Prueba Redarbor Developer



Objetivo:
Desarrollar una API RESTful que gestione un sistema de inventario de productos, implementando buenas prácticas de programación, patrones de diseño y principios SOLID. La solución debe estar dockerizada y utilizar una base de datos relacional (preferiblemente SQL Server).

Requisitos Generales:

- El código fuente debe ser publicado en un repositorio de código con acceso público (ej: GitHub, GitLab, bitbucket, etc.).
- Utilizar Entity Framework para operaciones de lectura y Dapper para operaciones de escritura.
- Implementar pruebas unitarias.
- Aplicar buenas prácticas de Programación Orientada a Objetos (POO), Test-Driven Development (TDD), SOLID y Clean Code.
- Implementar patrones de diseño, si es posible el patrón de diseño CQRS (Command Query Responsibility Segregation).
- Autenticación mediante oAuth2.
- El uso de herramientas de Inteligencia Artificial (IA) generativa (p. ej. GitHub Copilot, Claude Code, Cursor, etc.) está permitido y se alienta para asistir en la implementación. El candidato es responsable del 100% del código entregado: debe poder compilarlo, ejecutar las pruebas y defender cada decisión durante la entrevista. Es obligatorio declarar en el README qué herramienta(s) de IA se utilizaron, en qué fases y cómo se validó su salida.

Especificaciones Técnicas:

- Framework y Versión:
  - .NET 6.0 o .NET Core
- Base de Datos:
  - Utilizar una base de datos relacional SQL.
  - Diseñar un esquema de base de datos para gestionar productos e inventarios. Al menos debe incluir tablas para *Productos, Categorías* y *Movimientos de Inventario*.
    (Nota: No usar *FOREIGN KEYS* para donde vamos no necesitamos *FOREIGN KEYS*)
  - Dockerizar la base de datos para facilitar la configuración y despliegue.
- API RESTful:
  - Desarrollar endpoints para las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) de productos y categorías.
  - Implementar un endpoint para registrar movimientos de inventario (entrada/salida de productos).
  - Proteger los endpoints utilizando autenticación oAuth2.
- Dockerización:
  - Dockerizar la aplicación API y la base de datos para asegurar un despliegue consistente y fácil de configurar.
  - Proporcionar un archivo *docker-compose.yml* que permita levantar la aplicación y la base de datos con un solo comando.

- Pruebas Unitarias:
    - Implementar pruebas unitarias para los componentes críticos de la aplicación, asegurando una cobertura de código adecuada.
    - Creación de pruebas unitarias.
- Documentación:
    - Incluir un archivo *README.md* en el repositorio con instrucciones detalladas sobre cómo configurar, ejecutar y probar la aplicación (Tener en cuenta que se seguirán los pasos para desplegar y depurar la aplicación en local).
    - Documentar los endpoints de la API utilizando Swagger.
- Entrega:
    - El código fuente debe ser entregado a través de un repositorio público (ej: GitHub, GitLab, bitbucket, etc.).
    - Incluir en el repositorio todos los archivos necesarios para construir y ejecutar la aplicación y la base de datos utilizando Docker.

Uso de IA — Madurez en el proceso (Plus):

El uso de IA forma parte del flujo de trabajo esperado; el diferenciador no es consumir IA, sino orquestarla. Como plus se valora que el candidato demuestre un flujo de trabajo con IA estructurado y reproducible, con independencia de la herramienta que utilice. Como señal de madurez (no obligatorio) se acepta, por ejemplo:

- Un AGENTS.md o archivo de instrucciones que capture el stack, la arquitectura y las 8 reglas de Clean Code de esta prueba, para guiar a la IA durante la implementación.
- Un rol de revisión explícito: evidencia de que el candidato revisó la salida de la IA contra los estándares del proyecto (checklist, agente de code review, etc.).
- Un rol de verificación: build y pruebas corriendo, y una declaración de cómo se validó que el resultado cumple los requisitos.
- Un flujo documentado (propuesta → especificación/diseño → tareas → implementación → verificación), que puede ser Spec-Driven Development / OpenSpec (como se aplica en Redarbor) u otro flujo que el candidato estructure y justifique.
- Gestión eficiente del consumo de tokens: evidencia de que el candidato aplicó estrategias y/o herramientas para reducir el consumo de tokens del modelo (p. ej. proveer solo el contexto relevante, dividir el trabajo en tareas acotadas, evitar re-alimentar archivos completos, prompt caching o selección de modelo según la complejidad; y herramientas como memoria persistente entre sesiones tipo Engram, retrieval a nivel de símbolo tipo CodeGraph, o fetch de documentación bajo demanda tipo Context7).

Evaluación:

La prueba será evaluada en base a los siguientes criterios:

- Realizar una lectura de nuestras 8 reglas de Clean Code, pues esta prueba la evaluaremos con base en estos puntos.
- Cumplimiento de los requisitos funcionales y técnicos.
- Calidad del código y aplicación de buenas prácticas de programación.
- Diseño y arquitectura de la solución.
- Uso efectivo de patrones de diseño y principios SOLID.
- Cobertura y calidad de las pruebas unitarias.
- Uso efectivo y responsable de la IA: que el candidato la orqueste, la revise, la valide y siga un flujo reproducible (plus: workflow estructurado tipo SDD / skills / agents), incorporando estrategias para minimizar el consumo de tokens.

A continuación, se listan las reglas de Clean Code mínimas que se deben tener en la prueba:

1. Todo el código y comentarios están 100% en inglés, nada de español a excepción estricta de mensajes que se vayan a mostrar en respuesta.
2. No dejar espacios en blanco entre líneas de código a excepción de una línea antes del return.
3. Los nombres de los métodos y/o funciones deben ser autodescriptivos, no deberían requerir un comentario que explique su función.
4. Funciones con única responsabilidad, los principales indicadores de que no es de única responsabilidad son:
    a. Métodos y/o funciones con más de 25 líneas de código.
    b. Loops o condicionales anidados más de 1 vez.
    c. secciones de código dentro de la función que requieran comentarios para ser comprendidas
5. Las variables de configuración deben ser leídas de variables de entorno.
6. Tener un archivo por clase, es decir no poner más de una *CLASS* o *ENUM* en un solo archivo.
7. Pasar Objetos y no Propiedades como parámetros, en la medida de lo posible
    a. Bien: Product.Save(product)
    b. Mal: Product.Save(id, name, status, etc…)